

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES - ESPOCH

DOCUMENTO DE METÁFORA, ARQUITECTURA DIRIGIDA POR EVENTOS Y SELECCIÓN DEL STACK TECNOLÓGICO

Arquitectura Reactiva en Tiempo Real, Ecosistema de Herramientas y Justificación Técnica

Roles de Ingeniería:

- Líder Técnico y Arquitecto Backend
- Desarrollador Frontend y Diseñador UI/UX
- Especialista en Bases de Datos y QA Lead
- Ingeniero DevOps y Redes

Asignaturas y Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrutamiento: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Documento Oficial de Ingeniería de Software (Versión 2.0 - Arquitectura EDA)

Orellana - Ecuador

Noviembre de 2026

Índice general

1. Fundamentos y Metáfora del Sistema **4**

1.1. Metáfora Conceptual: Centro de Operaciones de Red (NOC) 4

2. Arquitectura Dirigida por Eventos (EDA) **5**

2.1. Estructura del Flujo Reactivo 5

2.2. Catálogo Estandarizado de Eventos 7

3. Desglose Técnico del Stack Tecnológico **8**

3.1. Componentes Seleccionados y Funciones 8

4. Matriz de Justificación Técnica y Alternativas Descartadas **9**

Índice de figuras

1.1. Diagrama de interacción general de los componentes del sistema 4

2.1. Diagrama de componentes y flujo de eventos del sistema (EDA) 6

3.1. Capas técnicas y distribución de responsabilidades del stack 8

Índice de tablas

2.1. Catálogo de eventos del bus interno 7

4.1. Matriz comparativa de arquitectura, tiempo real y desarrollo web 9

4.2. Matriz comparativa de persistencia, monitoreo de red e infraestructura . . . 10

Capítulo 1

Fundamentos y Metáfora del Sistema

1.1 Metáfora Conceptual: Centro de Operaciones de Red (NOC)

La metáfora del sistema proporciona un modelo mental claro y unificado del funcionamiento de la plataforma:

- **Consola de Visualización (Frontend):** Proyecta en pantallas interactivas el mapa de topología, indicadores clave de desempeño (KPIs) y el panel de alertas acústico-visuales.
- **Canal de Telemetría (WebSockets):** Enlace bidireccional continuo que transmite métricas y notificaciones en tiempo real sin requerir peticiones periódicas.
- **Centro de Control y Despacho (Backend API y Bus de Eventos):** Procesa peticiones administrativas seguras (autenticación JWT, gestión de usuarios, configuración) y gestiona el flujo asíncrono de eventos internos.
- **Motor de Sondeo (Servicio de Monitoreo):** Servicio autónomo en segundo plano que ejecuta comprobaciones periódicas de Capa 2 (ARP) y Capa 3 (ICMP ping) sobre los equipos registrados.
- **Almacén Central (MongoDB):** Repositorio documental que guarda inventario, métricas agregadas y bitácora histórica de eventos con políticas de retención automática (TTL).

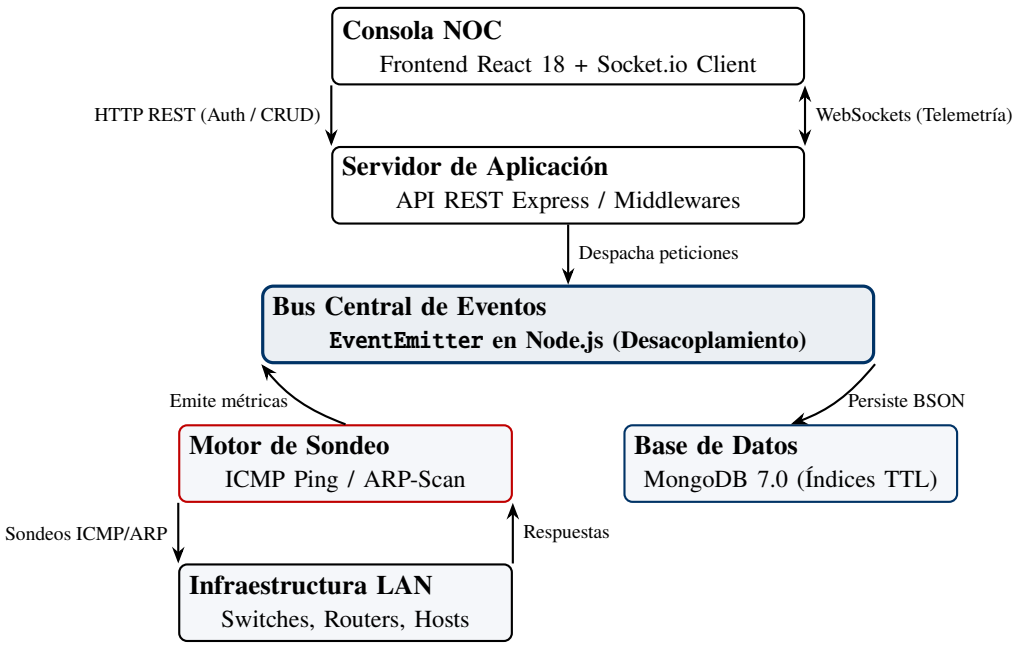


Figura 1.1: Diagrama de interacción general de los componentes del sistema

Capítulo 2

Arquitectura Dirigida por Eventos (EDA)

2.1 Estructura del Flujo Reactivo

El monitoreo de red opera sobre condiciones asíncronas: caídas de enlace, fluctuaciones de latencia e incorporación de nuevos hosts. La Arquitectura Dirigida por Eventos desacopla la detección de anomalías de su procesamiento y visualización, garantizando que el sistema reaccione en milisegundos sin bloquear el servidor web.

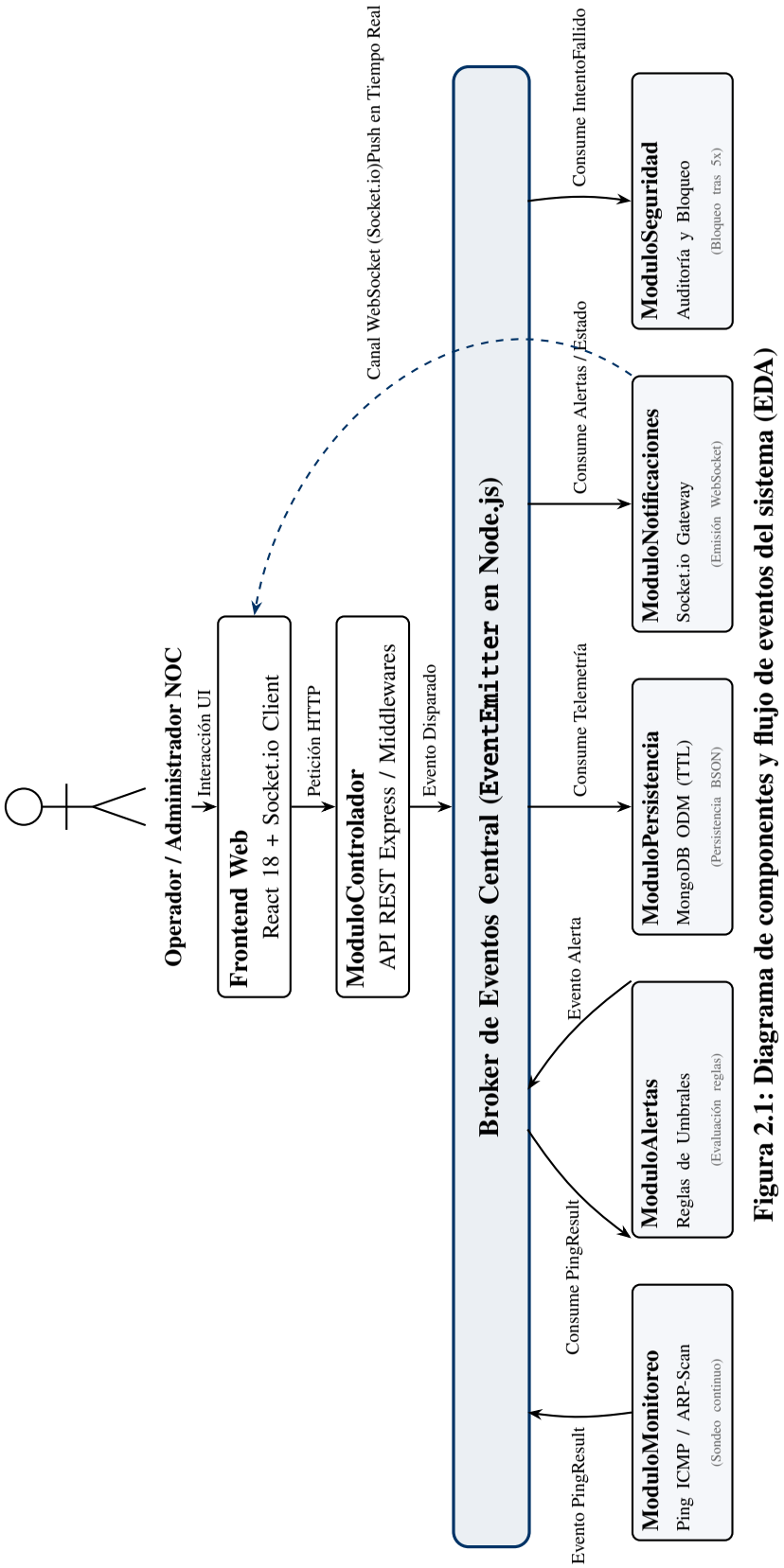


Figura 2.1: Diagrama de componentes y flujo de eventos del sistema (EDA)

2.2 Catálogo Estandarizado de Eventos

El bus de eventos opera mediante constantes inmutables para impedir el uso de cadenas mágicas y asegurar tipado estricto en el backend:

Tabla 2.1: Catálogo de eventos del bus interno

Evento Tipado	Productor	Consumidor(es)	Acción Desencadenada
`NETWORK_PING_RESULT`	ModuloMonitoreo	ModuloAlertas, ModuloPersistencia	Evalúa umbrales de latencia y persiste la telemetría en MongoDB.
`DEVICE_STATUS_CHANGED`	ModuloMonitoreo	ModuloAlertas, ModuloNotificaciones	Actualiza estado (`online`, `warning`, `offline`) y emite por WebSocket.
`DEVICE_DISCOVERED`	ModuloMonitoreo	ModuloNotificaciones	Notifica descubrimiento de un nuevo host no inventariado en el segmento LAN.
`ALERT_TRIGGERED`	ModuloAlertas	ModuloNotificaciones, ModuloPersistencia	Emite alerta visual y acústica al operador y crea el ticket en base de datos.
`AUTH_LOGIN_FAILED`	ModuloControlador	ModuloSeguridad	Incrementa contador de fallos; tras 5 intentos ejecuta bloqueo atómico temporal.
`TOPOLOGY_LINK_CHANGED`	ModuloControlador	ModuloNotificaciones	Actualiza aristas del grafo de red en tiempo real en la vista de topología.

Capítulo 3

Desglose Técnico del Stack Tecnológico

3.1 Componentes Seleccionados y Funciones

A continuación se detalla la función técnica específica de cada tecnología seleccionada:

- **React 18.2+ con Vite 5:** Interfaz reactiva concurrente. Actualiza el mapa de red y paneles métricos en tiempo real sin recargar la página.
- **Socket.io (v4.8 Backend / v4.7 Frontend):** Canal WebSocket persistente para transmisión bidireccional en tiempo real con reconexión automática y latencia inferior a 50 ms.
- **React Flow (@xyflow/react v12):** Motor de visualización para grafos interactivos de topología de red con nodos personalizados, aristas dinámicas, zoom y arrastre.
- **Node.js v20 LTS y Express v4.21+:** Entorno asíncrono no bloqueante con microframework para gestión de rutas REST, validación Zod y middlewares de seguridad (Helmet, CORS).
- **Bus de Eventos Interno (`EventEmitter`):** Despachador de mensajes en memoria que desacopla la captura de métricas, el motor de alertas y la persistencia.
- **Mecanismos de Red Nativos (`arp-scan` y `ping`):** Ejecución eficiente mediante procesos del kernel de Linux con pool de concurrencia (`p-limit` a 15 sockets simultáneos).
- **MongoDB 7.0 con Mongoose 8.9+:** Base de datos NoSQL documental con índices TTL para purga automática de sesiones y métricas históricas a 30 y 90 días.
- **Nginx 1.22+ y Systemd en Debian 12:** Servidor web inverso con terminación SSL y proxy WebSocket, orquestado como servicio nativo con política `Restart=always`.

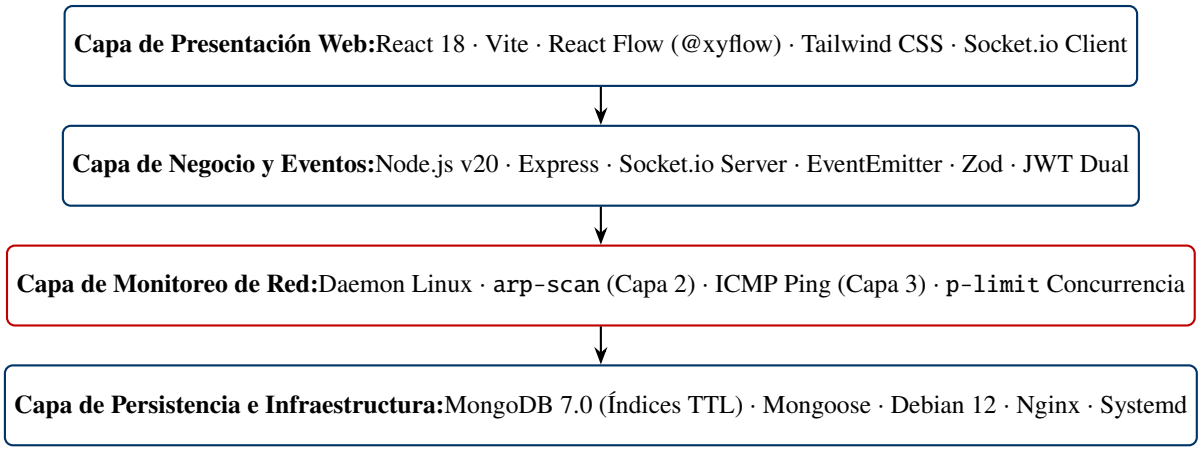


Figura 3.1: Capas técnicas y distribución de responsabilidades del stack

Capítulo 4

Matriz de Justificación Técnica y Alternativas Descartadas

Tabla 4.1: Matriz comparativa de arquitectura, tiempo real y desarrollo web

Capa / Componente	Tecnología Elegida	Alternativas Evaluadas	Criterio y Justificación de Decisión
Patrón Arquitectural	**Monolito Modular Dirigido por Eventos (EDA)**	Microservicios distribuidos, Monolito síncrono N-capas	Microservicios introduce sobrecarga de latencia de red entre servicios y complejidad innecesaria para un equipo de 4 personas. El monolito síncrono no puede responder en tiempo real a eventos de red. EDA modular ofrece el equilibrio óptimo entre desacoplamiento y simplicidad operativa.
Comunicación en Tiempo Real	**WebSockets (Socket.io)**	HTTP Polling periódico, Server-Sent Events (SSE)	HTTP Polling satura la red con peticiones vacías cada pocos segundos y genera latencia inaceptable. SSE solo permite flujo unidireccional servidor-cliente. Socket.io ofrece transporte bidireccional, reconexión inteligente, salas de aislamiento y latencia inferior a 50 ms.
Frontend Web	**React 18 + Vite**	Angular, Vue.js, Plantillas Server-Side (EJS/Django)	React cuenta con el ecosistema de visualización de grafos de red más maduro (@xyflow/react). Vite ofrece compilaciones ultrarrápidas y menor consumo de memoria en desarrollo que Webpack.
Backend & API	**Node.js + Express**	Python (Django/FastAPI), Java (Spring Boot)	El modelo de I/O asíncrono no bloqueante de Node.js es superior al manejar cientos de comprobaciones concurrentes de red sin levantar hilos pesados del sistema operativo.

Tabla 4.2: Matriz comparativa de persistencia, monitoreo de red e infraestructura

Capa / Componente	Tecnología Elegida	Alternativas Evaluadas	Criterio y Justificación de Decisión
Base de Datos	**MongoDB 7.0**	PostgreSQL, MySQL, SQLite	La estructura de los dispositivos de red es polimórfica (un router tiene tablas de rutas; un switch puertos y VLANs; una PC no). Los documentos BSON modelan esta jerarquía sin tablas relacionales artificiales. Dispone de índices TTL nativos.
Daemon de Red	**Node.js Daemon + arp-scan/ping**	Python Scapy, C/C++ raw sockets	‘arp-scan‘ y ‘ping‘ son binarios del sistema altamente optimizados por el kernel de Linux. Integrarlos con Node.js proporciona equilibrio entre velocidad de ejecución en bajo nivel y facilidad de lógica de negocio.
Despliegue	**Systemd + Nginx en Debian**	Docker Compose en VM, Kubernetes	En la máquina física del laboratorio asignada para el proyecto, capas pesadas de virtualización consumen memoria RAM innecesaria. La ejecución como servicio nativo Systemd garantiza el 100 % del rendimiento del hardware.